

PROGRAMACIÓN IV

Trabajo Práctico - Api Rest

Desarrollo de una API REST completa y profesional para la gestión de productos

Aplicando arquitectura en capas, validaciones, manejo de errores, persistencia con Spring Data JPA y documentación con Swagger.

Alumno:

Matias Carro - matiasmanuelcarro@gmail.com

DNI: 37362003

Legajo: 18743

API REST Spring Boot.....	2
1. Descripción general del proyecto.....	2
Trabajo Práctico - Api Rest.....	2
2. Dependencias principales.....	2
Spring Boot.....	2
Swagger (OpenAPI).....	2
3. Arquitectura del proyecto.....	3
Entities.....	5
DTOs (Data Transfer Objects).....	5
Repository.....	6
Service.....	7
Controller.....	7
AdviceController (Manejo de errores).....	7
4. Carga inicial de datos (CommandLineRunner).....	8
5. Pruebas con Postman.....	8
Crear datos (POST).....	8
Crear usuarios.....	8
Crear categorías.....	12
Crear pedidos con detalles.....	27
Actualizar categoría.....	33
Buscar por ID.....	35
Buscar por mail.....	35
6. Swagger funcionando.....	36

API REST Spring Boot

1. Descripción general del proyecto

Trabajo Práctico - Api Rest

OBJETIVO GENERAL

- Desarrollar una API REST completa y profesional para la gestión de productos
 - Aplicar arquitectura en capas
 - Implementar validaciones
 - Implementar manejo de errores
 - Implementar persistencia con Spring Data JPA
 - Implementar documentación con Swagger
-

2. Dependencias principales

Spring Boot

- Spring Web
- Spring Data JPA
- Lombok
- H2 Database
- Spring Boot DevTools

Swagger (OpenAPI)

```
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.8.8'
```

Swagger UI esta disponible en:

<http://localhost:8080/swagger-ui.html>

3. Arquitectura del proyecto

El proyecto sigue una arquitectura en capas:

```
|-- controller/

|   |-- CategoriaController.java
|   |-- UsuarioController.java
|   |-- ProductoController.java
|   |-- PedidoController.java
|
|-- service/

|   |-- UsuarioService.java
|   |-- ProductoService.java
|   |-- CategoriaService.java
|   |-- PedidoService.java
|
|-- repository/

|   |-- UsuarioRepository.java
|   |-- ProductoRepository.java
|   |-- CategoriaRepository.java
|   |-- PedidoRepository.java
|
|-- entities/

|   |-- Usuario.java
|   |-- Producto.java
```

```
|    |-- Categoria.java

|    |-- Pedido.java

|    |-- DetallePedido.java

|

|-- dtos/

|
|    |-- usuario/
|
|        |-- UsuarioDto.java
|
|        |-- UsuarioCreate.java
|
|        |-- UsuarioEdit.java
|
|
|    |-- producto/
|
|        |-- ProductoDto.java
|
|        |-- ProductoCreate.java
|
|        |-- ProductoEdit.java
|
|
|    |-- categoria/
|
|        |-- CategoriaDto.java
|
|        |-- CategoriaCreate.java
|
|        |-- CategoriaEdit.java
|
|
|-- pedido/

    |-- PedidoDto.java
```

```
| -- PedidoCreate.java  
  
| -- PedidoEdit.java  
  
| -- PedidoCreateConUsuario.java
```

Cada capa tiene una responsabilidad clara.

Entities

Representan las tablas de la base de datos.

- Usuario
 - Producto
 - Categoria
 - Pedido
 - DetallePedido
-

DTOs (Data Transfer Objects)

Los DTOs permiten:

- Controlar qué datos se exponen
- Validar entradas
- Evitar exponer entidades directamente

DTOS del proyecto:

- **usuario/**
 - UsuarioDto.java
 - UsuarioCreate.java
 - UsuarioEdit.java
- **producto/**
 - ProductoDto.java
 - ProductoCreate.java
 - ProductoEdit.java
- **categoria/**
 - CategoriaDto.java

- CategoriaCreate.java
 - CategoriaEdit.java
 - **pedido/**
 - PedidoDto.java
 - PedidoCreate.java
 - PedidoEdit.java
 - PedidoCreateConUsuario.java
-

Repository

Cada entidad tiene su repositorio:

Usuario

```
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {}
```

Producto

```
public interface ProductoRepository extends JpaRepository<Producto, Long> {}
```

Categoría

```
public interface CategoriaRepository extends JpaRepository<Categoria, Long> {}
```

Pedido

```
public interface PedidoRepository extends JpaRepository<Pedido, Long> {}
```

Detalle Pedido

```
public interface DetallePedidoRepository extends JpaRepository<DetallePedido, Long> {}
```

Los repositorios permiten:

- Guardar entidades
- Buscar por ID
- Buscar por atributos
- Listar todos
- Eliminar

Service

La capa Service implementa la lógica de negocio.

- Validaciones
- Conversión entre DTO y entidad
- Manejo de excepciones
- Reglas de negocio (soft delete, asignación de pedidos, etc.)

Los servicios del documento:

- UsuarioService
- ProductoService
- CategoriaService
- PedidoService

Cada servicio usa su repositorio y devuelve DTOs.

Controller

Los controladores exponen la API REST.

Ejemplos del documento:

- UsuarioController
- ProductoController
- PedidoController
- CategoriaController

Cada controlador implementa:

- GET (listar, buscar por ID, buscar por mail)
 - POST (crear)
 - PUT (editar)
 - DELETE (eliminar o soft delete)
-

AdviceController (Manejo de errores)

Se implementa un `@RestControllerAdvice` para capturar excepciones:

- EntityNotFound
- Validaciones
- Datos inválidos

- Errores de negocio

Ejemplo:

```
@ExceptionHandler(EntityNotFoundException.class)

public ResponseEntity<?> handleNotFound(EntityNotFoundException ex)
```

4. Carga inicial de datos (CommandLineRunner)

El documento incluye un CommandLineRunner que crea datos para pruebas:

- 5 usuarios
 - 5 categorías
 - 10 productos
 - 10 pedidos con detalles
 - Asignación de pedidos a usuarios
-

5. Pruebas con Postman

Crear datos (POST)

Crear usuarios

Usuario 1 POST /usuarios

```
{
  "nombre": "Carolina",
  "apellido": "Gutierrez",
  "mail": "carolina.g@mail.com",
  "celular": "1167894321",
  "password": "caro123",
  "rol": "USUARIO"
}
```

API REST > ReuquestBody

POST `http://localhost:8080/usuarios`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "nombre": "Carolina",
3   "apellido": "Gutierrez",
4   "mail": "carolina.g@mail.com",
5   "telefono": "1167894321",
6   "password": "caro123",
7   "celular": "116286245",
8   "rol": "USUARIO"
9 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "id": 6,
3   "nombre": "Carolina",
4   "apellido": "Gutierrez",
5   "mail": "carolina.g@mail.com",
6   "celular": "116286245",
7   "rol": "USUARIO"
8 }
```

▼ 5:

```
id: 6
nombre: "Carolina"
apellido: "Gutierrez"
mail: "carolina.g@mail.com"
celular: "116286245"
rol: "USUARIO"
```

Usuario 2 POST /usuarios

```
{
  "nombre": "Roberto",
  "apellido": "Sanchez",
  "mail": "roberto.s@mail.com",
  "celular": "1176543210",
  "password": "rober123",
  "rol": "ADMIN"
}
```

API REST > ReuquestBody

POST `http://localhost:8080/usuarios`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "nombre": "Roberto",
3   "apellido": "Sanchez",
4   "mail": "roberto.s@mail.com",
5   "celular": "1176543210",
6   "password": "rober123",
7   "rol": "ADMIN"
8 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "id": 7,
3   "nombre": "Roberto",
4   "apellido": "Sanchez",
5   "mail": "roberto.s@mail.com",
6   "celular": "1176543210",
7   "rol": "ADMIN"
8 }
```

```
▼ 6:
  id:      7
  nombre:  "Roberto"
  apellido: "Sanchez"
  mail:    "roberto.s@mail.com"
  celular: "1176543210"
  rol:     "ADMIN"
```

Usuarios completos:

```
▼ 0:
  id:      1
  nombre:   "Matías"
  apellido: "Carro"
  mail:     "matias@mail.com"
  celular:  "1155555555"
  rol:      "USUARIO"
▼ 1:
  id:      2
  nombre:   "Gisel"
  apellido: "Castro"
  mail:     "gisel@mail.com"
  celular:  "1144444444"
  rol:      "USUARIO"
▼ 2:
  id:      3
  nombre:   "Luis"
  apellido: "Gomez"
  mail:     "luis@mail.com"
  celular:  "1133333333"
  rol:      "USUARIO"
▼ 3:
  id:      4
  nombre:   "German"
  apellido: "Ruiz"
  mail:     "german@mail.com"
  celular:  "1122222222"
  rol:      "USUARIO"
```

```
▼ 4:
  id:      5
  nombre:   "Jorge"
  apellido: "Ramírez"
  mail:     "jorge@mail.com"
  celular:  "1111111111"
  rol:      "ADMIN"
▼ 5:
  id:      6
  nombre:   "Carolina"
  apellido: "Gutierrez"
  mail:     "carolina.g@mail.com"
  celular:  "116286245"
  rol:      "USUARIO"
▼ 6:
  id:      7
  nombre:   "Roberto"
  apellido: "Sanchez"
  mail:     "roberto.s@mail.com"
  celular:  "1176543210"
  rol:      "ADMIN"
```

Crear categorías

Categoría 1 POST /categorias

```
{
  "nombre": "Redes",
  "descripcion": "Routers, switches, placas de red"
}
```

The screenshot shows the API REST client interface. At the top, it displays 'API REST > ReuquestBody' (note the typo). The request method is 'POST' and the URL is 'http://localhost:8080/categorias'. The 'Body' tab is selected, showing the raw JSON request body:

```
{
  "nombre": "Redes",
  "descripcion": "Routers, switches, placas de red"
}
```

. Below the request, the response is shown with a status of '200 OK'. The response body is displayed in JSON format:

```
{
  "id": 6,
  "nombre": "Redes",
  "descripcion": "Routers, switches, placas de red",
  "eliminado": false
}
```

```
▼ 5:
  id:      6
  nombre:  "Redes"
  descripcion: "Routers, switches, placas de red"
  eliminado: false
```

Categoría 2 POST /categorias

```
{
  "nombre": "Impresion",
  "descripcion": "Impresoras, tintas, repuestos"
}
```

API REST > ReuquestBody

POST http://localhost:8080/categorias

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nombre": "Impresion",
3   "descripcion": "Impresoras, tintas, repuestos"
4 }
```

Body Cookies Headers 5 Test Results 200 OK • 5 t

JSON Preview Visualize

```
1 {
2   "id": 7,
3   "nombre": "Impresion",
4   "descripcion": "Impresoras, tintas, repuestos",
5   "eliminado": false
6 }
```

```
6:
  id:      7
  nombre:  "Impresion"
  descripcion: "Impresoras, tintas, repuestos"
  eliminado: false
```

Categoría 3 POST /categorias

```
{
  "nombre": "Sillas Gamer",
  "descripcion": "Sillas ergonómicas y gamer"
}
```

HTTP API REST > ReuquestBody

POST ▼ `http://localhost:8080/categorias`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "nombre": "Sillas Gamer",
3   "descripcion": "Sillas ergonómicas y gamer"
4 }
```

Body Cookies Headers 5 Test Results ↺ **200 OK** • 5

{} JSON ▼ ▶ Preview 🖼️ Visualize ▼

```
1 {
2   "id": 8,
3   "nombre": "Sillas Gamer",
4   "descripcion": "Sillas ergonómicas y gamer",
5   "eliminado": false
6 }
```

▼ 7:

```
id: 8
nombre: "Sillas Gamer"
descripcion: "Sillas ergonómicas y gamer"
eliminado: false
```

Categorías Completas:

▼ 0:	
id:	1
nombre:	"Periféricos"
descripcion:	"Teclados, mouse, auriculares"
eliminado:	false
▼ 1:	
id:	2
nombre:	"Almacenamiento"
descripcion:	"Discos SSD, HDD, memorias"
eliminado:	false
▼ 2:	
id:	3
nombre:	"Componentes"
descripcion:	"Placas de video, motherboards, procesadores"
eliminado:	false
▼ 3:	
id:	4
nombre:	"Monitores"
descripcion:	"Pantallas LED, IPS, 144Hz"
eliminado:	false
▼ 4:	
id:	5
nombre:	"Accesorios"
descripcion:	"Cables, adaptadores, soportes"
eliminado:	false

▼ 5:	
id:	6
nombre:	"Redes"
descripcion:	"Routers, switches, placas de red"
eliminado:	false
▼ 6:	
id:	7
nombre:	"Impresion"
descripcion:	"Impresoras, tintas, repuestos"
eliminado:	false
▼ 7:	
id:	8
nombre:	"Sillas Gamer"
descripcion:	"Sillas ergonómicas y gamer"
eliminado:	false

Productos de redes

POST /productos

```
{
  "nombre": "Router TP-Link AX1800",
  "precio": 52000,
  "descripcion": "WiFi 6",
  "stock": 25,
  "imagen": "router.jpg",
  "activo": true,
  "idCategoria": 1
}
```

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:8080/productos
- Body Type:** raw
- Request Body (JSON):**

```
{
  "nombre": "Router TP-Link AX1800",
  "precio": 52000,
  "descripcion": "WiFi 6",
  "stock": 25,
  "imagen": "router.jpg",
  "activo": true,
  "idCategoria": 1
}
```
- Response Status:** 201 Created
- Response Body (JSON):**

```
{
  "id": 11,
  "nombre": "Router TP-Link AX1800",
  "precio": 52000,
  "descripcion": "WiFi 6",
  "stock": 25,
  "imagen": "router.jpg",
  "disponible": false
}
```

```
▼ 10:
  id:      11
  nombre:   "Router TP-Link AX1800"
  precio:   52000.00 JS: 52000
  descripcion: "WiFi 6"
  stock:    25
  imagen:    "router.jpg"
  disponible: false
```

```
{
  "nombre": "Switch Gigabit 8 Puertos",
  "precio": 35000,
  "descripcion": "1000 Mbps",
  "stock": 40,
  "imagen": "switch.jpg",
  "activo": true,
  "idCategoria": 1
}
```

POST http://localhost:8080/productos

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "nombre": "Switch Gigabit 8 Puertos",
3   "precio": 35000,
4   "descripcion": "1000 Mbps",
5   "stock": 40,
6   "imagen": "switch.jpg",
7   "activo": true,
8   "idCategoria": 1
9 }
```

Body Cookies Headers 5 Test Results 201 Created 5

{ } JSON Preview Visualize

```
1 {
2   "id": 12,
3   "nombre": "Switch Gigabit 8 Puertos",
4   "precio": 35000,
5   "descripcion": "1000 Mbps",
6   "stock": 40,
7   "imagen": "switch.jpg",
8   "disponible": false
9 }
```

```
▼ 11:
  id:      12
  nombre:  "Switch Gigabit 8 Puertos"
  precio:  35000.00 JS: 35000
  descripcion: "1000 Mbps"
  stock:    40
  imagen:   "switch.jpg"
  disponible: false
```

```
{
  "nombre": "Placa de Red PCIe",
  "precio": 15000,
  "descripcion": "Gigabit Ethernet",
  "stock": 60,
  "imagen": "nic.jpg",
  "activo": true,
  "idCategoria": 1
}
```

POST http://localhost:8080/productos

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

```
1 {
2   "nombre": "Placa de Red PCIe",
3   "precio": 15000,
4   "descripcion": "Gigabit Ethernet",
5   "stock": 60,
6   "imagen": "nic.jpg",
7   "activo": true,
8   "idCategoria": 1
9 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 13,
3   "nombre": "Placa de Red PCIe",
4   "precio": 15000,
5   "descripcion": "Gigabit Ethernet",
6   "stock": 60,
7   "imagen": "nic.jpg",
8   "disponible": false
9 }
```

```

▼ 12:
  id:      13
  nombre:  "Placa de Red PCIe"
  precio:  15000.00 JS: 15000
  descripcion: "Gigabit Ethernet"
  stock:   60
  imagen:  "nic.jpg"
  disponible: false

```

Productos de Impresión

```

{
  "nombre": "Impresora HP DeskJet 2776",
  "precio": 68000,
  "descripcion": "Multifunción WiFi",
  "stock": 15,
  "imagen": "hp.jpg",
  "activo": true,
  "idCategoria": 2
}

```

POST http://localhost:8080/productos

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```

1 {
2   "nombre": "Impresora HP DeskJet 2776",
3   "precio": 68000,
4   "descripcion": "Multifunción WiFi",
5   "stock": 15,
6   "imagen": "hp.jpg",
7   "activo": true,
8   "idCategoria": 2
9 }

```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```

1 {
2   "id": 14,
3   "nombre": "Impresora HP DeskJet 2776",
4   "precio": 68000,
5   "descripcion": "Multifunción WiFi",
6   "stock": 15,
7   "imagen": "hp.jpg",
8   "disponible": false
9 }

```

```
▼ 13:
  id:      14
  nombre:   "Impresora HP DeskJet 2776"
  precio:   68000.00 JS: 68000
  descripcion: "Multifunción WiFi"
  stock:    15
  imagen:    "hp.jpg"
  disponible: false
```

```
{
  "nombre": "Cartucho Negro HP 428",
  "precio": 12000,
  "descripcion": "Original",
  "stock": 80,
  "imagen": "cartucho.jpg",
  "activo": true,
  "idCategoria": 2
}
```

POST <http://localhost:8080/productos>

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "nombre": "Cartucho Negro HP 428",
3   "precio": 12000,
4   "descripcion": "Original",
5   "stock": 80,
6   "imagen": "cartucho.jpg",
7   "activo": true,
8   "idCategoria": 2
9 }
```

Body Cookies Headers 5 Test Results **201 Created**

{ } JSON ▼ Preview Visualize ▼

```
1 {
2   "id": 15,
3   "nombre": "Cartucho Negro HP 428",
4   "precio": 12000,
5   "descripcion": "Original",
6   "stock": 80,
7   "imagen": "cartucho.jpg",
8   "disponible": false
9 }
```

```
▼ 14:
  id:      15
  nombre:   "Cartucho Negro HP 428"
  precio:   12000.00 JS:12000
  descripcion: "Original"
  stock:    80
  imagen:    "cartucho.jpg"
  disponible: false
```

```
{
  "nombre": "Resma A4 80g",
  "precio": 6000,
  "descripcion": "500 hojas",
  "stock": 100,
  "imagen": "resma.jpg",
  "activo": true,
  "idCategoria": 2
}
```

POST <http://localhost:8080/productos>

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "nombre": "Resma A4 80g",
3   "precio": 6000,
4   "descripcion": "500 hojas",
5   "stock": 100,
6   "imagen": "resma.jpg",
7   "activo": true,
8   "idCategoria": 2
9 }
```

Body Cookies Headers 5 Test Results  **201 Created**

{} JSON ▼ Preview Visualize ▼

```
1 {
2   "id": 16,
3   "nombre": "Resma A4 80g",
4   "precio": 6000,
5   "descripcion": "500 hojas",
6   "stock": 100,
7   "imagen": "resma.jpg",
8   "disponible": false
9 }
```

```
▼ 15:
  id:      16
  nombre:   "Resma A4 80g"
  precio:   6000.00 JS: 6000
  descripcion: "500 hojas"
  stock:    100
  imagen:   "resma.jpg"
  disponible: false
```

Productos de Sillas Gamer

```
{
  "nombre": "Silla Gamer Razer",
  "precio": 150000,
  "descripcion": "Ergonómica premium",
  "stock": 10,
  "imagen": "razer.jpg",
  "activo": true,
  "idCategoria": 3
}
```

POST `http://localhost:8080/productos`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1  {
2    "nombre": "Silla Gamer Razer",
3    "precio": 150000,
4    "descripcion": "Ergonómica premium",
5    "stock": 10,
6    "imagen": "razer.jpg",
7    "activo": true,
8    "idCategoria": 3
9  }

```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON ▾ ▶ Preview 🖼 Visualize ▾

```

1  {
2    "id": 17,
3    "nombre": "Silla Gamer Razer",
4    "precio": 150000,
5    "descripcion": "Ergonómica premium",
6    "stock": 10,
7    "imagen": "razer.jpg",
8    "disponible": false
9  }

```

▼ 16:

```

id: 17
nombre: "Silla Gamer Razer"
precio: 150000.00 JS:150000
descripcion: "Ergonómica premium"
stock: 10
imagen: "razer.jpg"
disponible: false

```

```

{
  "nombre": "Silla Gamer Redragon",
  "precio": 110000,
  "descripcion": "Cuero sintético",
  "stock": 20,
  "imagen": "redragon.jpg",
  "activo": true,
  "idCategoria": 3
}

```

POST `http://localhost:8080/productos`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1  {
2      "nombre": "Silla Gamer Redragon",
3      "precio": 110000,
4      "descripcion": "Cuero sintético",
5      "stock": 20,
6      "imagen": "redragon.jpg",
7      "activo": true,
8      "idCategoria": 3
9  }

```

Body Cookies Headers 5 Test Results 201 Created

{} JSON ▾ Preview Visualize ▾

```

1  {
2      "id": 18,
3      "nombre": "Silla Gamer Redragon",
4      "precio": 110000,
5      "descripcion": "Cuero sintético",
6      "stock": 20,
7      "imagen": "redragon.jpg",
8      "disponible": false
9  }

```

▼ 17:

id:	18
nombre:	"Silla Gamer Redragon"
precio:	110000.00 JS: 110000
descripcion:	"Cuero sintético"
stock:	20
imagen:	"redragon.jpg"
disponible:	false

```

{
  "nombre": "Silla Gamer Corsair",
  "precio": 130000,
  "descripcion": "Soporte lumbar",
  "stock": 12,
  "imagen": "corsair.jpg",

```

```
"activo": true,  
"idCategoria": 3  
}
```

POST http://localhost:8080/productos

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {  
2   "nombre": "Silla Gamer Corsair",  
3   "precio": 130000,  
4   "descripcion": "Soporte lumbar",  
5   "stock": 12,  
6   "imagen": "corsair.jpg",  
7   "activo": true,  
8   "idCategoria": 3  
9 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {  
2   "id": 19,  
3   "nombre": "Silla Gamer Corsair",  
4   "precio": 130000,  
5   "descripcion": "Soporte lumbar",  
6   "stock": 12,  
7   "imagen": "corsair.jpg",  
8   "disponible": false  
9 }
```

18:

id:	19
nombre:	"Silla Gamer Corsair"
precio:	130000.00 JS:130000
descripcion:	"Soporte lumbar"
stock:	12
imagen:	"corsair.jpg"
disponible:	false

```
{
  "nombre": "Almohadilla Lumbar",
  "precio": 9000,
  "descripcion": "Memory foam",
  "stock": 50,
  "imagen": "almohadilla.jpg",
  "activo": true,
  "idCategoria": 3
}
```

POST http://localhost:8080/productos

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "nombre": "Almohadilla Lumbar",
3   "precio": 9000,
4   "descripcion": "Memory foam",
5   "stock": 50,
6   "imagen": "almohadilla.jpg",
7   "activo": true,
8   "idCategoria": 3
9 }
```

Body Cookies Headers 5 Test Results 201 Created

{ } JSON Preview Visualize

```
1 {
2   "id": 20,
3   "nombre": "Almohadilla Lumbar",
4   "precio": 9000,
5   "descripcion": "Memory foam",
6   "stock": 50,
7   "imagen": "almohadilla.jpg",
8   "disponible": false
9 }
```

```
▼ 18:
  id:      19
  nombre:   "Silla Gamer Corsair"
  precio:   130000.00 JS:130000
  descripcion: "Soporte lumbar"
  stock:    12
  imagen:    "corsair.jpg"
```

Crear pedidos con detalles

POST /pedidos

Pedido 1 (Usuario 1)

```
{
  "fecha": "2026-08-29",
  "estado": "PENDIENTE",
  "formaPago": "EFECTIVO",
  "usuarioId": 1,
  "detalles": [
    { "cantidad": 1, "productoId": 1 },
    { "cantidad": 2, "productoId": 4 }
  ]
}
```

POST ▼ <http://localhost:8080/pedidos>

☰ Docs Params Authorization Headers 8 **Body** ● Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "fecha": "2026-08-29",
3   "estado": "PENDIENTE",
4   "formaPago": "EFECTIVO",
5   "usuarioId": 1,
6   "detalles": [
7     { "cantidad": 1, "productoId": 1 },
8     { "cantidad": 2, "productoId": 4 }
9   ]
10 }
```

Body Cookies Headers 5 Test Results 🕒 200 OK

{ } JSON ▼ ▶ Preview 🖼️ Visualize ▼

```
1 {
2   "id": 11,
3   "fecha": "2026-08-29",
4   "estado": "PENDIENTE",
5   "total": 89000.00,
6   "formapago": null,
7   "detalles": [
8     {
9       "id": 15,
10      "cantidad": 2,
11      "subtotal": 64000.00,
12      "productoId": 4,
13      "productoNombre": "SSD Kingston 480GB"
14    },
15    {
16      "id": 16,
17      "cantidad": 1,
18      "subtotal": 25000.00,
19      "productoId": 1,
20      "productoNombre": "Teclado Mecánico Redragon"
21    }
22  ]
23 }
```

▼ 10:	
id:	11
fecha:	"2026-08-29"
estado:	"PENDIENTE"
total:	89000.00 JS: 89000
formapago:	null
▼ detalles:	
▼ 0:	
id:	15
cantidad:	2
subtotal:	64000.00 JS: 64000
productoId:	4
productoNombre:	"SSD Kingston 480GB"
▼ 1:	
id:	16
cantidad:	1
subtotal:	25000.00 JS: 25000
productoId:	1
productoNombre:	"Teclado Mecánico Redragon"

Pedido 2 (Usuario 2)

```
{
  "fecha": "2026-08-29",
  "estado": "CONFIRMADO",
  "formaPago": "TARJETA",
  "usuarioId": 2,
  "detalles": [
    { "cantidad": 3, "productoId": 7 },
    { "cantidad": 1, "productoId": 10 }
  ]
}
```

POST ▼ http://localhost:8080/pedidos

Docs Params Authorization Headers 8 **Body** ● Scripts Settings

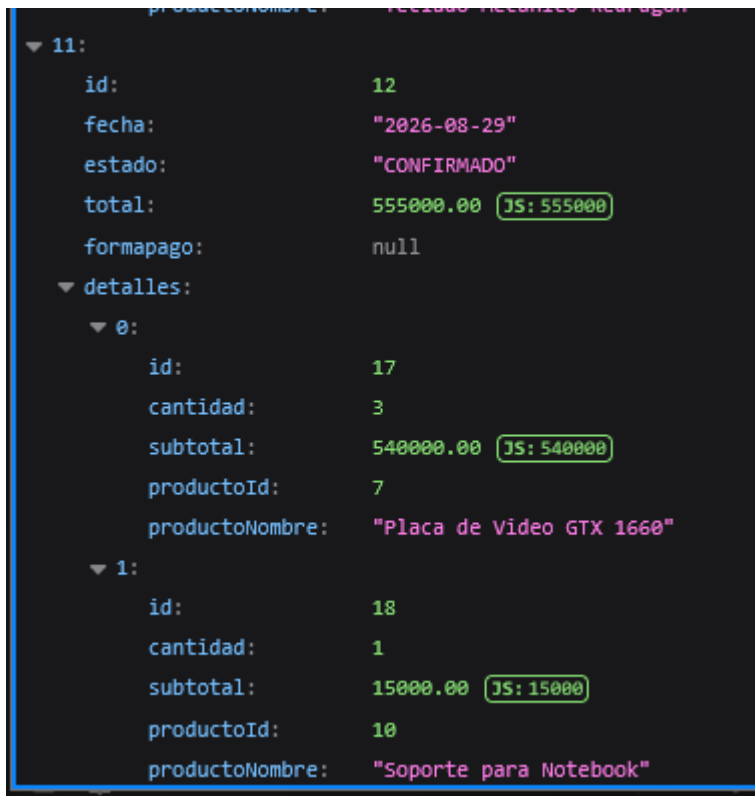
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼

```
1 {
2   "fecha": "2026-08-29",
3   "estado": "CONFIRMADO",
4   "formaPago": "TARJETA",
5   "usuarioId": 2,
6   "detalles": [
7     { "cantidad": 3, "productoId": 7 },
8     { "cantidad": 1, "productoId": 10 }
9   ]
10 }
```

Body Cookies Headers 5 Test Results ↺ 200 OK

{ } JSON ▼ ▶ Preview 🔍 Visualize ▼

```
1 {
2   "id": 12,
3   "fecha": "2026-08-29",
4   "estado": "CONFIRMADO",
5   "total": 555000.00,
6   "formapago": null,
7   "detalles": [
8     {
9       "id": 17,
10      "cantidad": 3,
11      "subtotal": 540000.00,
12      "productoId": 7,
13      "productoNombre": "Placa de Video GTX 1660"
14    },
15    {
16      "id": 18,
17      "cantidad": 1,
18      "subtotal": 15000.00,
19      "productoId": 10,
20      "productoNombre": "Soporte para Notebook"
21    }
22  ]
23 }
```



Pedido 3 (Usuario 1)

```
{
  "fecha": "2026-08-29",
  "estado": "TERMINADO",
  "formaPago": "TRANSFERENCIA",
  "usuarioId": 1,
  "detalles": [
    { "cantidad": 2, "productId": 3 },
    { "cantidad": 1, "productId": 9 }
  ]
}
```

POST `http://localhost:8080/pedidos`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```
1 {
2   "fecha": "2026-08-29",
3   "estado": "TERMINADO",
4   "formaPago": "TRANSFERENCIA",
5   "usuarioId": 1,
6   "detalles": [
7     { "cantidad": 2, "productoId": 3 },
8     { "cantidad": 1, "productoId": 9 }
9   ]
10 }
```

Body Cookies Headers 5 Test Results 200 OK

{ } **JSON** ▾ ▶ Preview 🖼 Visualize ▾

```
1 {
2   "id": 13,
3   "fecha": "2026-08-29",
4   "estado": "TERMINADO",
5   "total": 95000.00,
6   "formapago": null,
7   "detalles": [
8     {
9       "id": 19,
10      "cantidad": 2,
11      "subtotal": 90000.00,
12      "productoId": 3,
13      "productoNombre": "Auriculares HyperX Cloud II"
14    },
15    {
16      "id": 20,
17      "cantidad": 1,
18      "subtotal": 5000.00,
19      "productoId": 9,
20      "productoNombre": "Cable HDMI 2.0"
21    }
22  ]
23 }
```



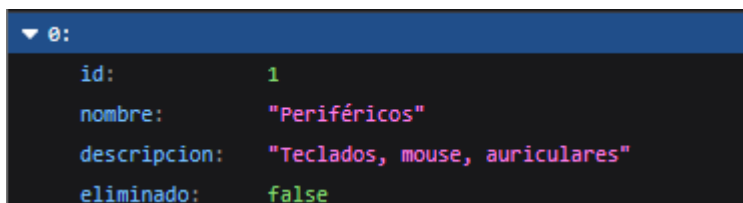
Actualizar categoría

PUT /categorias/{id}

Body:

```
{ "nombre": "Periféricos Gamer", "descripcion": "Teclados y mouse de alta gama" }
```

Antes:



Actualización:

PUT `http://localhost:8080/categorias/1`

Docs Params Authorization Headers 8 **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON ▾

1 { "nombre": "Periféricos Gamer", "descripcion": "Teclados y mouse de alta gama" }

Body Cookies Headers 5 Test Results 200 OK • 21

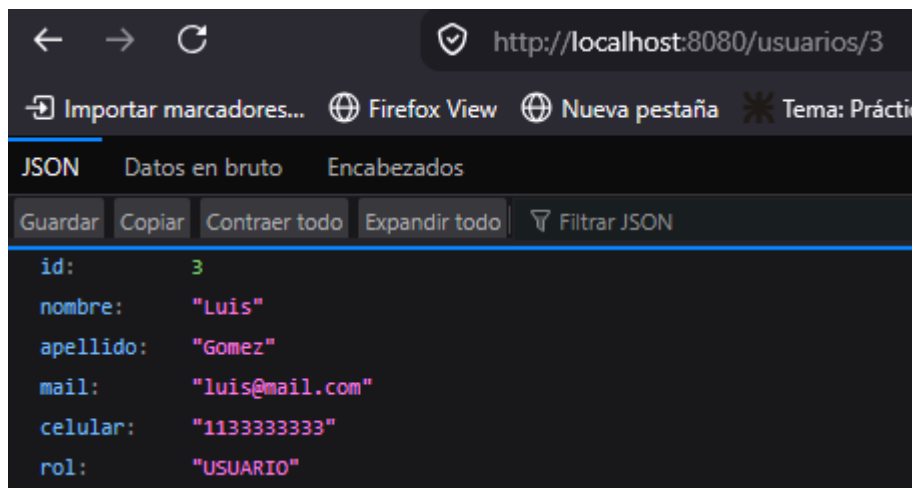
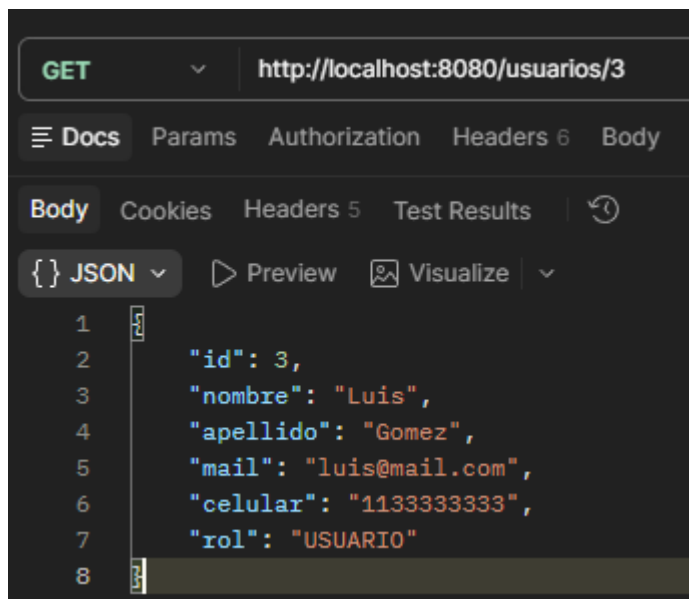
{ } JSON ▾ Preview Visualize ▾

```
1 {
2   "id": 1,
3   "nombre": "Periféricos Gamer",
4   "descripcion": "Teclados y mouse de alta gama",
5   "eliminado": false
6 }
```

```
▼ 0:
  id:      1
  nombre:  "Periféricos Gamer"
  descripcion: "Teclados y mouse de alta gama"
  eliminado: false
```

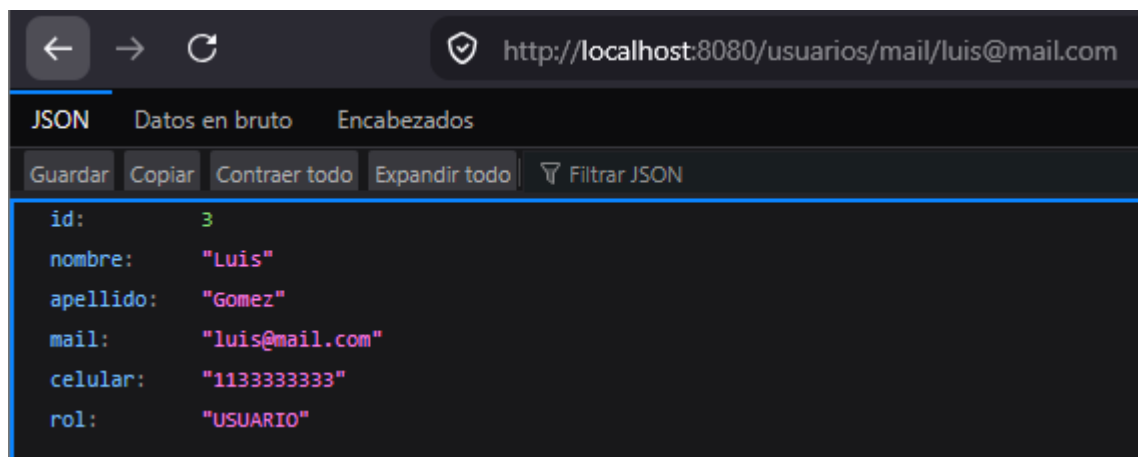
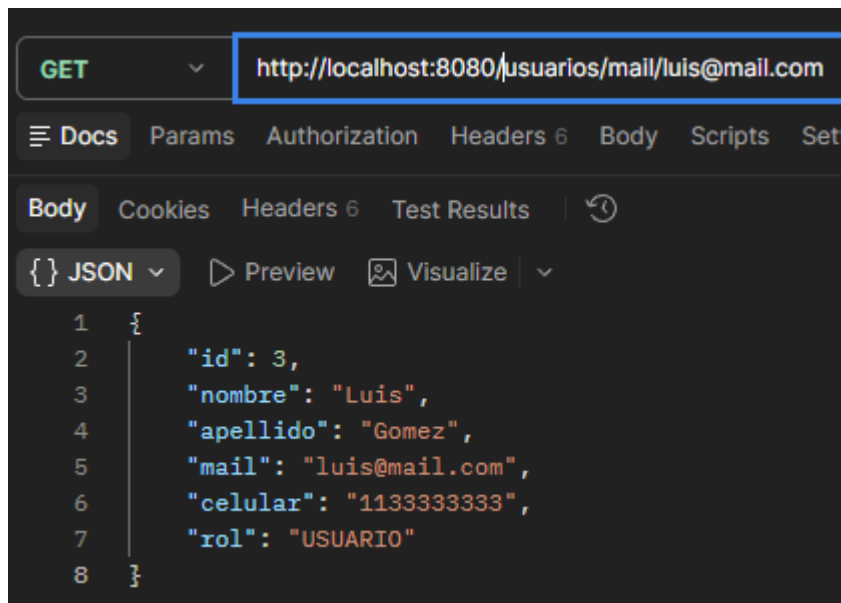
Buscar por ID

GET /usuarios/{id}



Buscar por mail

GET /usuarios/mail/{mail}



6. Swagger funcionando

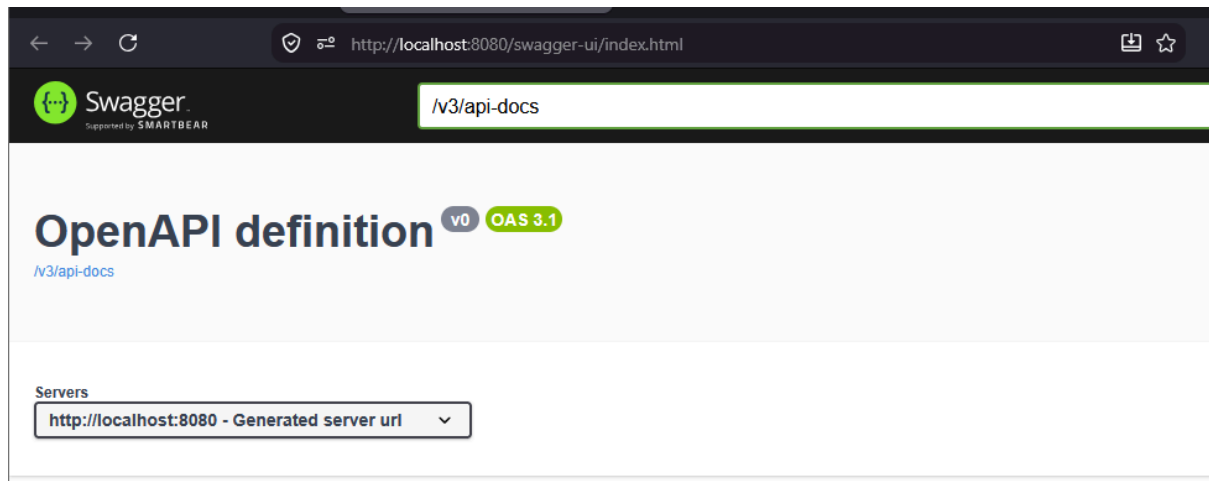
Acceder a:

<http://localhost:8080/swagger-ui.html>

Swagger muestra:

- Todos los endpoints
- DTOs
- Métodos HTTP
- Ejemplos de request y response

Swagger responde correctamente:



Usuario Controller

usuario-controller		^
GET	/usuarios/{id}	✓
PUT	/usuarios/{id}	✓
DELETE	/usuarios/{id}	✓
GET	/usuarios	✓
POST	/usuarios	✓
GET	/usuarios/{id}/nombre	✓
GET	/usuarios/mail/{mail}	✓

usuario-controller

GET

/usuarios/{id}

Parameters

Try it out

Name	Description
id * required	
integer(\$int64)	id
(path)	

Responses

Code	Description	Links
200	OK	No links

Media type

/

Controls Accept header.

Example Value | Schema

```
{
  "id": 9007199254740991,
  "nombre": "string",
  "apellido": "string",
  "mail": "string",
  "celular": "string",
  "rol": "ADMIN"
}
```

Producto Controller

producto-controller

GET

/productos/{id}

PUT

/productos/{id}

DELETE

/productos/{id}

GET

/productos

POST

/productos

GET

/productos/eliminados

Pedido Controller

pedido-controller

GET

/pedidos/{id}

PUT

/pedidos/{id}

DELETE

/pedidos/{id}

GET

/pedidos

POST

/pedidos

POST

/pedidos/asignar

Categoría Controller

categoria-controller		^
GET	/categorias/{id}	▼
PUT	/categorias/{id}	▼
DELETE	/categorias/{id}	▼
GET	/categorias	▼
POST	/categorias	▼

Schemas:

Schemas

UsuarioEdit ^ Collapse all object

nombre string
apellido string
mail string
celular string
contrasenia string
rol > Expand all string

UsuarioDto ^ Collapse all object

id integer int64
nombre string
apellido string
mail string
celular string
rol > Expand all string

ProductoEdit ^ Collapse all object

nombre string
precio number
descripcion string
stock integer int32
imagen string
disponible boolean
idCategoria integer int64

ProductoDto ^ Collapse all object

id integer int64
nombre string
precio number
descripcion string
stock integer int32
imagen string
disponible boolean

PedidoEdit ^ Collapse all object

fecha string **date**
estado > Expand all string
formapago > Expand all string

DetallePedidoDto ^ Collapse all object

id integer **int64**
cantidad integer **int32**
subtotal number
productoId integer **int64**
productoNombre string

PedidoDto ^ Collapse all object

id integer **int64**
fecha string **date**
estado > Expand all string
total number
formapago > Expand all string
detalles > Expand all array<object>

CategoriaEdit ^ Collapse all object

nombre string
descripcion string

CategoriaDto ^ Collapse all object

id integer **int64**
nombre string
descripcion string
eliminado boolean

UsuarioCreate ^ Collapse all object

nombre string
apellido string
mail string
celular string
contrasenia string
rol > Expand all string

ProductoCreate ^ Collapse all object

nombre string
precio number
descripcion string
stock integer int32
imagen string
disponible boolean
idCategoria integer int64

DetallePedidoCreate ^ Collapse all object

cantidad integer int32
productoId integer int64

PedidoCreate ^ Collapse all object

fecha string date
estado > Expand all string
formapago > Expand all string
detalles > Expand all array<object>

PedidoCreateConUsuario ^ Collapse all object

usuarioId integer **int64**

pedido > Expand all object

CategoriaCreate ^ Collapse all object

nombre string

descripcion string

UsuarioNombreDto ^ Collapse all object

nombre string

apellido string